

BIRLA INSTITUTE OF TECHNOLOGY, MESRA, RANCHI

Department of Production & Industrial Engineering

CAN WE TRUST THE AI AGENT?

A trust layer for AI agents making supply-chain decisions

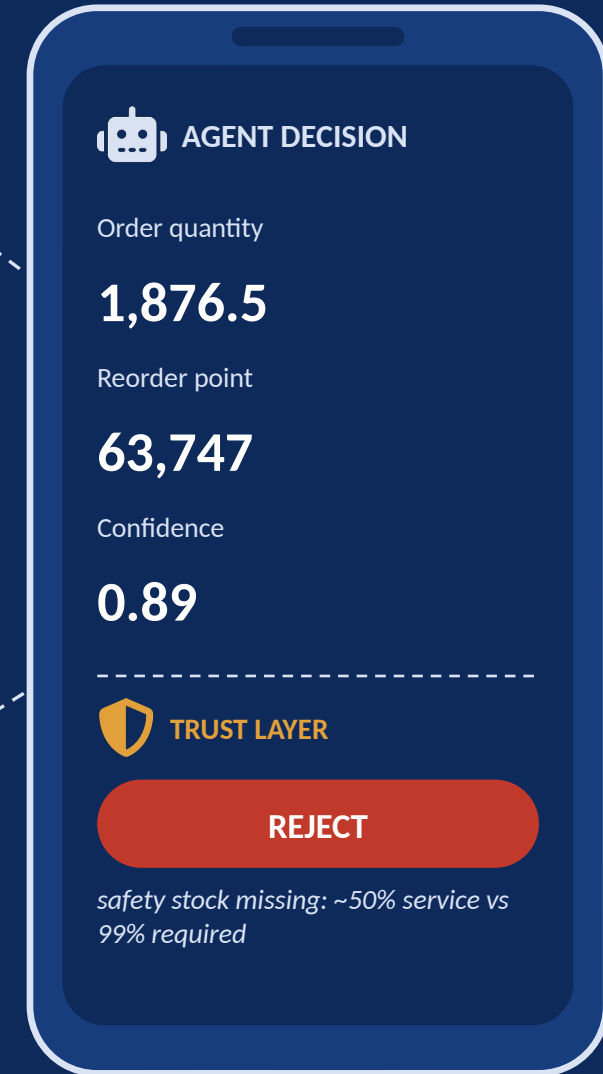
MINOR PROJECT • PROGRESSIVE EVALUATION • VII SEMESTER • 2026-27

PRESENTED BY

- Student Name 1 (Roll No.)
- Student Name 2 (Roll No.)
- Student Name 3 (Roll No.)

UNDER THE GUIDANCE OF

Guide Name
Designation
Dept. of PIE
BIT Mesra, Ranchi





Introduction: The Trust Problem

Companies are trying AI agents for planning decisions, but a wrong answer looks exactly like a right one

WHAT IS HAPPENING

Decisions being handed to AI agents



How much to order?



When to reorder?



Which supplier?

Why this is risky

- 1 An LLM always gives an answer. A wrong order quantity is printed in the same format as a right one, often with the same confidence.
- 2 Checking every decision by hand defeats the point of automating it.
- 3 In real operations there is no answer key to compare against.

Goal

Tell which agent outputs are safe to accept and which need a planner, without knowing the correct answer.

AN EXAMPLE FROM OUR BENCHMARK

Problem eoq-0000 (adversarial version)



861,765

units / year



27 days

lead time



99%

service target

Answer A

Order quantity: **1,876.5**

Reorder point: **70,673**

Answer B

Order quantity: **1,876.5**

Reorder point: **63,747**

Which one would you approve?



B is the agent's answer (stated confidence 0.89). It left out the safety stock of 6,926 units, so it gives only about **50% service instead of 99%**.

Nothing in the output itself warns you.



Project Overview

Operations-research problems have provably optimal answers, so an AI agent can be graded exactly



3

Decision families



108

Decisions per run



5

Trust checks



4

Seeds tested

TWO QUESTIONS WE ASK

Q1

How often is the AI agent wrong on decisions whose optimum is known?
Measured with the answer key (scoring only).

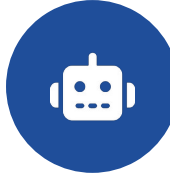
Q2

Can we catch the wrong decisions without knowing the correct answer?
Our main contribution: the trust layer.

WHAT WE BUILT IN PHASE 1



Benchmark
108 problems per run, each with a verified optimum
`problems.py`



AI Agent
Real LLM (API) or an offline simulator
`agent.py`



Trust Layer
5 checks → ACCEPT / ESCALATE / REJECT
`verifier.py`



Scoring
Grades agent and verifier; dashboard
`evaluate.py`

Team of 3:

Module A · Benchmark

Module B · AI Agent

Module C · Trust + scoring



Classical Models Behind Our Answer Key

Standard inventory and LP results give an exact optimum for every benchmark problem



Ref.	Work	Key idea	How we use it
[1]	Harris (1913), Economic Order Quantity	Balancing ordering cost against holding cost gives $Q^* = \sqrt{(2DS/H)}$.	Ground truth for the EOQ family; at Q^* yearly ordering cost = yearly holding cost, reused as a check.
[2]	Arrow, Harris & Marschak (1951), Optimal inventory policy	Optimal stock under random demand depends on shortage and overage costs.	Basis of the newsvendor critical-ratio answer, $Q^* = \mu + z\sigma$.
[3]	Silver, Pyke & Thomas (2017), inventory textbook	Reorder point = lead-time demand + safety stock $z \cdot \sigma \cdot \sqrt{L}$.	Ground-truth reorder point and the service-level check.
[4]	Huangfu & Hall (2018), HiGHS LP solver	Fast dual revised simplex for exact LP solutions.	Solves the sourcing LP via SciPy linprog(method="highs").
[5]	MNRE ALMM Order (2019), Govt. of India	Only listed solar PV models/makers allowed in government-backed projects.	Motivates the domestic-supplier floor (floor values in our problems are synthetic).



These models give an exact optimum for every problem in our benchmark, so grading the agent needs no human judgement.



LLMs Applied to Optimisation & Supply Chains

Most work tries to make the LLM produce a better answer, and checks it offline against labelled data

2021 Training Verifiers for Math Word Problems <i>[9] Cobbe et al. (OpenAI)</i> What they did Released GSM8K (8.5K problems). A separately trained verifier ranks candidate solutions, which improves accuracy. For our problem Verification helps, but the verifier is trained on labelled solutions.	2022 NL4Opt Competition <i>[7] Ramamonjison et al.</i> What they did Turning plain-English optimisation problems into LP formulations; first dataset of LP word problems. For our problem Accuracy is measured offline against labelled formulations.	2023 OptiGuide: LLMs for Supply Chain <i>[6] Li et al. (Microsoft)</i> What they did LLM turns a planner's what-if question into code; a real solver computes the answer. Used on Microsoft's cloud supply chain. For our problem Industry interest is real. Correctness comes from the solver, not the LLM.	2024 OptiMUS <i>[8] AhmadiTeshnizi et al.</i> What they did LLM agent that formulates MILP models from text, writes and debugs solver code, and checks the solutions. For our problem Aims at a better model; runs a solver every time.
---	--	---	--

What we noticed

→ These works measure accuracy offline against labelled answers, or keep a solver inside the loop. Checking the agent's final numbers at run time, when no answer key exists, gets much less attention.



How Reliable Are LLM Outputs?

Distraction, over-confidence and failed self-correction: four findings that shaped our design



Distracted by irrelevant text

[10] Shi et al., ICML 2023

Adding irrelevant sentences to math word problems (GSM-IC) clearly lowers LLM accuracy.

In our project

Every problem has a distractor version with irrelevant numbers (floor area, SKUs, freight budget).



Over-confident in words

[11] Xiong et al., ICLR 2024

Confidence that LLMs state in words tends to be too high; methods struggle on tasks needing professional knowledge.

In our project

An answer is never accepted because of its confidence; confidence only escalates a flagged answer.



Weak at self-correction

[12] Huang et al., ICLR 2024

Without outside feedback LLMs often cannot fix their own reasoning; self-correction can make answers worse.

In our project

Checking is done outside the agent, by plain rule-based Python code.



Poorly calibrated

[13] Guo et al., ICML 2017

Modern neural networks' confidence often does not match how often they are actually right.

In our project

We measure the agent's confidence when right vs when wrong as a separate result.



What Is Missing

The work we reviewed does not check an agent's supply-chain decision at run time without an answer key



THE GAP

A simple, explainable check that sits between an AI agent and a supply-chain decision, and decides ACCEPT / ESCALATE / REJECT without knowing the correct answer.

Tested on problems where the true optimum is known, so the check itself can be graded.

Label-free

Run-time

Explainable

EXISTING APPROACH

01

Offline accuracy only

Accuracy is measured against an answer key [7], [9]. In production there is no answer key.

02

Trained verifiers

Need labelled solutions, or rely on an LLM that can also be wrong [9], [12].

03

Confidence taken as a signal

Wrong answers come with high stated confidence [11], [13].

04

Re-solving every decision

LLM + solver pipelines re-run the full model each time [6], [8].

WHAT OUR PROJECT DOES

- Verifier sees only problem data + agent response
- Answer key used only for scoring

- Five rule-based checks from OR theory
- No training; every flag can be explained

- Confidence never used to accept
- Only escalates an answer that failed a check

- Necessary conditions + simple baselines
- The optimum is never recomputed



Objectives of the Work

Six objectives, each mapped to one module and to code that exists in the repository



1

Build the benchmark

Supply-chain decisions with provably optimal answers: EOQ + reorder point, newsvendor, LP sourcing with a domestic floor; base, distractor and adversarial versions.

Module A

Done



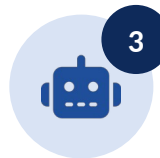
2

Verify the answer key

Re-derive every optimum with a second method: grid search, Monte Carlo simulation, random feasible allocations.

Module A

Done



3

Build the AI agent

A real LLM agent through the Anthropic API, plus an offline simulator that reproduces known LLM mistakes.

Module B

LLM run pending



4

Design the trust layer

Flag wrong decisions without seeing the answer key, and output ACCEPT / ESCALATE / REJECT.

Module C

Done (5 checks)



5

Score agent and verifier

Agent accuracy; verifier recall, precision, ROC-AUC and error rate among accepted answers.

Module C

Done



6

Report and extend

One-page dashboard now; 6 problem families and real-LLM results by the final evaluation.

Module C / all

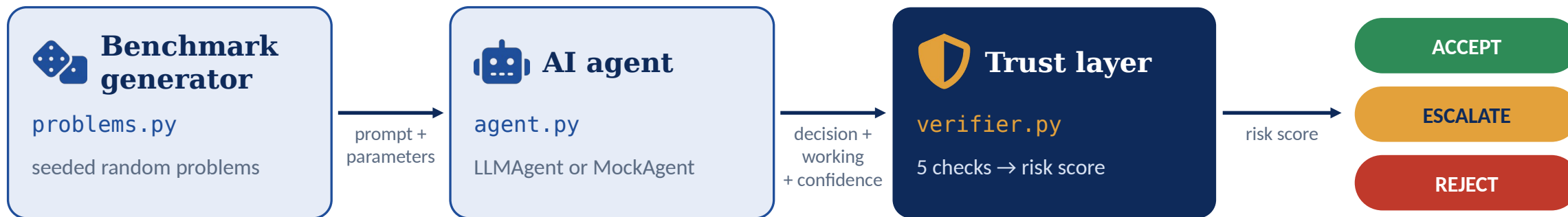
Partly done



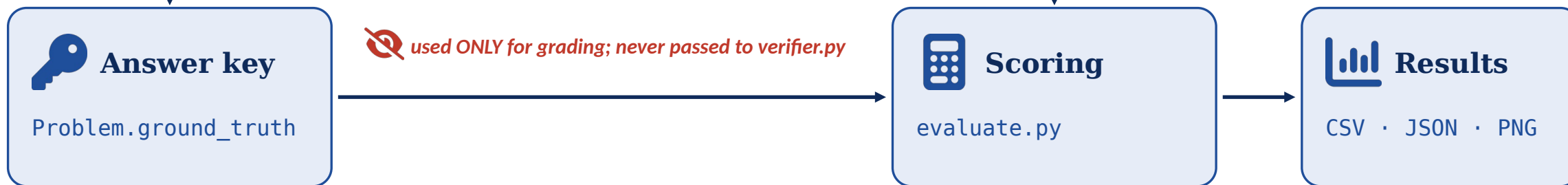
System Architecture

The verifier sees only what a real deployment would see; the answer key is used only for grading

PRODUCTION SIDE · what a real deployment would also have



EVALUATION SIDE · only possible because the optimum is known



`run_benchmark.py` loops: generate → decide → audit → score. Python · NumPy · SciPy [14] · anthropic SDK [15] · scikit-learn [16] · Matplotlib



Module A: Benchmark and Ground Truth

Three decision families, each with a provably optimal answer and three versions of the question



Family 1

EOQ + Reorder Point

How much to order, and when to reorder?

$$Q^* = \sqrt{(2DS / H)}$$
$$ROP = \mu_d L + z \sigma_d \sqrt{L}$$
$$z = \Phi^{-1}(\text{service level})$$

 Closed form (Harris + safety stock)



Family 2

Newsvendor

How many units to order once for the season?

$$CR = C_u / (C_u + C_o)$$
$$C_u = p - c, \quad C_o = c - s$$
$$Q^* = \mu + z \sigma, \quad z = \Phi^{-1}(CR)$$

 Closed form (critical ratio)



Family 3

LP Sourcing (ALMM/DCR)

How much to buy from each supplier?

$$\min \sum_i c_i x_i$$
$$\sum_i x_i = \text{demand}, \quad 0 \leq x_i \leq \text{cap}_i$$
$$\sum_{\text{dom}} x_i \geq \text{floor} \times \text{demand}$$

 SciPy linprog, HiGHS [4], [14]

THREE VERSIONS OF EVERY PROBLEM



Base

clean statement



Distractor

irrelevant numbers added



Adversarial

wording pushes a known mistake

No external dataset · seeded random parameters · $3 \times 12 \times 3 = 108$ decisions per run

ANSWER KEY CHECKED INDEPENDENTLY

0.0004%

EOQ vs grid search

200,000 points

0.25%

Newsvendor vs MC

Monte Carlo, 200,000 draws

0 cheaper

LP vs random plans

5,000 feasible allocations

max error vs formula · tests/test_ground_truth.py: 108 / 108 pass



Module B: The AI Agent

One interface, two agents: a real LLM and an offline simulator that makes realistic mistakes



MISTAKES THE SIMULATOR REPRODUCES

Family	Mode (in code)	What goes wrong
EOQ	<code>no_safety_stock</code>	ROP = lead-time demand only
EOQ	<code>eoq_drop_2</code>	Uses $\sqrt{(DS/H)}$: Q ~29% too small
Newsvendor	<code>inverted_cr</code>	Orders $\mu - z\sigma$ instead of $\mu + z\sigma$
Newsvendor	<code>ignore_salvage</code>	Treats salvage value as 0
LP sourcing	<code>greedy_cheapest / ignore_dcr</code>	Cheapest first; ignores the domestic floor

30 / 22 / 55%

mistake rate: base /
distractor / adversarial

N(0.91, 0.03)

confidence, right AND
wrong answers [11]

Pending

real-LLM run (Phase 2)



Module C: The Trust Layer

Five rule-based checks, none of which uses the answer key; the highest severity decides the outcome



Structural

Is it a valid answer? Present, finite, non-negative; one value per supplier.

Severity
1.0



Feasibility

EOQ: does the ROP give the target service level? LP: demand, capacities, domestic floor met?

Severity
0.9 / 1.0



Internal consistency

Does the answer follow from the agent's own working? (CR > 0.5 but orders below mean; zero safety stock at 99%)

Severity
0.8 - 0.9



Economic dominance

EOQ: ordering vs holding cost ratio ≤ 1.3 . Newsvendor: not >3% worse than ordering the mean. LP: not costlier than a simple plan.

Severity
0.4 - 0.9



Calibration

If a check is above 0.3 and confidence ≥ 0.85 , add $0.15 \times$ confidence.

Severity
+ bump

COMBINING THE CHECKS

risk = max(severity of all checks)
max, not average: one hard violation can't be outvoted



verify() in verifier.py

```
report.risk_score = max(c.severity for c in report.checks)
if report.risk_score < ACCEPT_BELOW:      # 0.33
    report.decision = "ACCEPT"
elif report.risk_score > REJECT_ABOVE:    # 0.66
    report.decision = "REJECT"
else: report.decision = "ESCALATE"
```



Work Completed So Far

About 1,300 lines of Python across 7 files; the full pipeline runs end to end

MODULE A · BENCHMARK

 problems.py (244)

 test_ground_truth.py (106)

- 3 families × 3 versions, seeded generator
- Answer key verified for all 108 problems

Member: (name)

MODULE B · AI AGENT

 agent.py (230)

 run_benchmark.py (100)

- MockAgent with 6 mistake modes
- LLMAgent: API call + JSON parsing
- CLI runner writes CSV + JSON

Member: (name)

MODULE C · TRUST + SCORING

 verifier.py (324)

 evaluate.py (153)

 dashboard.py (142)

- 5 checks, risk score, decision
- Accuracy, recall, precision, ROC-AUC
- One-page dashboard

Member: (name)

terminal · our runs (shortened)

```
$ python tests/test_ground_truth.py
eq: 36 OK | newsvendor: 36 OK | lp_sourcing: 36 OK
ALL GROUND-TRUTH CHECKS PASSED.
$ python src/run_benchmark.py --agent mock --seed 42
Agent accuracy 80.6% | Error rate AMONG ACCEPTED 0.0%
Verifier recall 100.0% | TP=21 FP=1 FN=0
```

Can We Trust the AI Agent? — Trust Layer for Supply-Chain Decisions

agent = mock | 360 decisions across EQQ, Newsvendor, and LPSOURCING (offline simulation)

Error rate BEFORE
19%
of the agent's raw decisions

Error rate AMONG ACCEPTED
0%
after the trust layer

Errors caught (recall)
100%
precision 95%

ROC-AUC of risk score
0.993
separates right from wrong

The agent is just as confident when it is WRONG
→ its confidence can't police itself → an external verifier is needed

The risk score cleanly separates right from wrong

What the trust layer did with each decision

Which check caught the errors (a caught error can trip more than one)

Dashboard

made by dashboard.py from the run's CSV and JSON



Trust Layer in Action

Three real decisions from our seed-42 run, and what the verifier did with each one

CASE 1 · EOQ, ADVERSARIAL

eoq-0000-adversarial

AGENT SAID

Order qty 1,876.5, ROP 63,747
Confidence 0.89

CHECKS THAT FIRED

Feasibility 0.90: ROP gives only 50% service vs 99% required

Internal 0.80: safety stock ≈ 0 but target is 99%

Calibration 1.00: confident on a flagged answer

REJECT

risk 1.00

True ROP 70,673: the safety stock of 6,926 was dropped.

CASE 2 · LP SOURCING

lp_sourcing-0005-base

AGENT SAID

Cheapest first: 2,954 | 5,474 | 0 | 16,873 | 5,410
Confidence 0.94

CHECKS THAT FIRED

Feasibility 1.00: domestic content 35% < required 40%

Calibration 1.00: confident on a flagged answer

REJECT

risk 1.00

Costs Rs 79.14 lakh, LESS than the true optimum (Rs 79.51 lakh): cheaper only because it breaks the rule.

CASE 3 · EOQ, BASE

eoq-0001-base

AGENT SAID

Order qty 7,726.6, ROP 106,651
Confidence 0.90

CHECKS THAT FIRED

Dominance 0.50: ordering Rs 5.09 lakh vs holding Rs 2.55 lakh: ratio 2.00

Calibration 0.64: bump for high confidence

ESCALATE

risk 0.64

True EOQ 10,927.1: the agent dropped the 2 in $\sqrt{(2DS/H)}$.

Output of verifier.verify(). The yellow boxes use the answer key and are only for explanation; the verifier never saw them.



Results: Seed-42 Run

108 decisions from the offline simulator (MockAgent); real-LLM results come in Phase 2



80.6%
Agent accuracy
87 of 108 correct



19.4%
Raw error rate
21 of 108 wrong



0.0%
Errors among accepted
0 wrong in 86 accepted



100%
Recall
21 of 21 wrong flagged



95.5%
Precision
21 of 22 flags correct

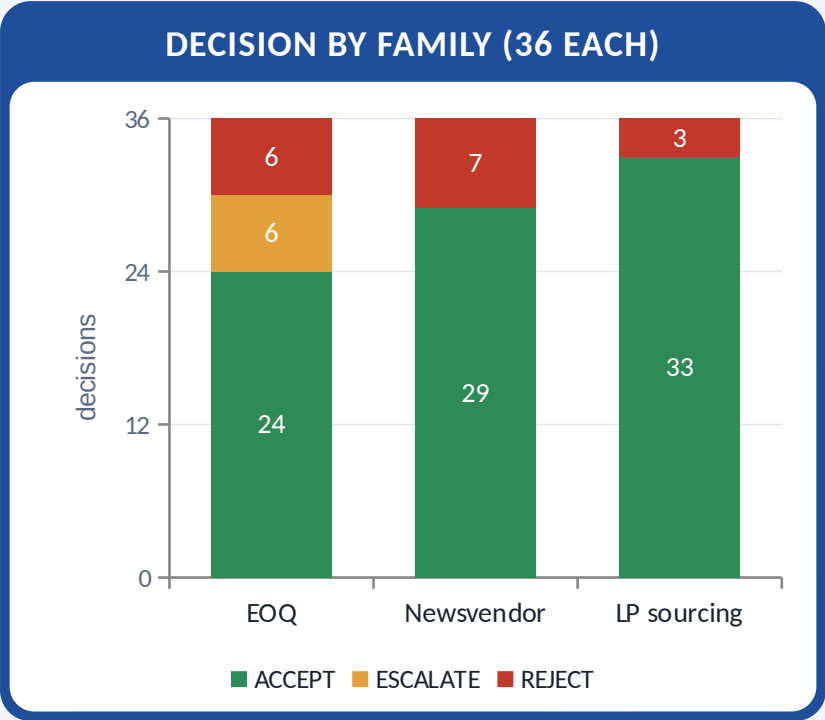


0.993
ROC-AUC [17]
separates right from wrong

CONFUSION MATRIX

	Flagged	Accepted
Agent wrong	21 caught	0 missed
Agent right	1 false alarm	86 correctly accepted

Correct = within 5% of optimal cost/profit AND constraints met (evaluate.py)



SAME TEST, 4 SEEDS

Seed	Accuracy	Recall	Precision	AUC
42	80.6%	100%	95.5%	0.993
7	79.6%	100%	100.0%	1.000
99	78.7%	100%	88.5%	0.978
2024	81.5%	100%	90.9%	0.985

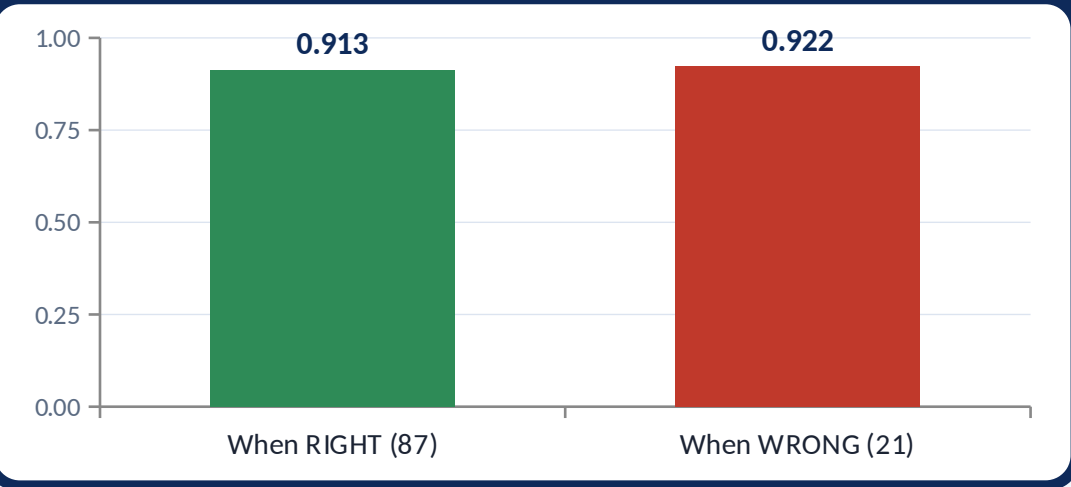
 **Errors among accepted = 0% in all four runs (108 decisions each).**



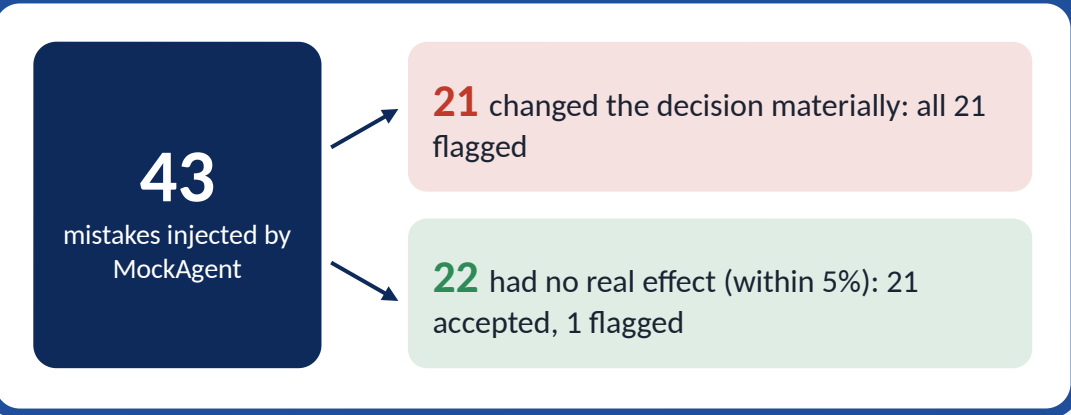
Key Findings So Far

The agent's own confidence tells us almost nothing; the trust layer's checks do

AGENT'S MEAN CONFIDENCE (SEED 42)



INJECTED MISTAKES VS REAL ERRORS



WHAT THE RESULTS MEAN

- 1 Confidence is useless as a filter.**
The agent was 0.913 confident when right and 0.922 when wrong, so the check has to come from outside the agent.
- 2 Every wrong answer was caught in all 4 runs.**
Recall 100% and 0% errors among accepted for seeds 42, 7, 99, 2024; false alarms 0 to 3 per 108 decisions.
- 3 The verifier judges the decision, not the method.**
22 injected mistakes did not change the outcome (e.g. ignoring salvage stayed within 5% profit); 21 of these were correctly accepted.
- 4 False alarms have a clear cause.**
All 6 (across 4 runs) were 'inverted critical ratio' answers with CR 0.60–0.63: ordered on the wrong side of the mean, but losing only 3–5% profit.
- 5 Severity follows the evidence.**
Rule breaks (service level, domestic floor) went to REJECT; a cost imbalance (dropped 2 in EOQ) went to ESCALATE at about 0.63.



Limitations and Our Planned Fixes

We name our own gaps first; each one has a concrete Phase-2 fix

LIMITATION NOW		PLAN FOR PHASE 2	
01	All results so far come from the offline simulator (MockAgent), not a real LLM.	→	Run LLMAgent (ready: --agent llm) on the same benchmark; report real-LLM numbers separately.
02	100% recall is on mistakes we planted, and the verifier was designed knowing them.	→	Test one mistake type the verifier was NOT designed for (an honesty test).
03	Only single-item, single-period problems (EOQ, newsvendor, one-stage sourcing).	→	Add multi-period lot sizing (Wagner-Whitin [18]), two-echelon base-stock [19], transportation.
04	Thresholds (0.33 / 0.66) and margins (1.3 ratio, 3%, 0.15) are set by judgement.	→	Tune thresholds on a held-out set of problems, then report on unseen problems.
05	MockAgent always uses seed 123, so the 4 runs repeat the same pattern of injected mistakes.	→	Vary the agent seed and use larger n; real-LLM runs remove this issue.
06	Economic checks need a hand-made baseline for each family.	→	Write baselines for new families; add a 6th check: robustness to small input changes.



Phase-2 Plan (till the Final Evaluation)

Bigger problems, a real LLM and a stronger verifier, one work package per module



Module A

Bigger Problem Families

What we will do

Add multi-period lot sizing, two-echelon base-stock and transportation, each with a provable optimum and tests.

Why it matters

Real planning is multi-period and multi-stage; 3 single-period families are not enough.

3 families → 6



Module B

Real LLM Runs

What we will do

Run LLMAgent on all families and seeds; study the real mistakes it makes.

Why it matters

All current numbers are from the simulator; the real test is an actual LLM.

Simulator → real LLM



Module C

Stronger Verifier

What we will do

Add a 6th robustness check, tune thresholds on held-out data, add Rs cost impact to the dashboard.

Why it matters

Thresholds are hand-set today; the verifier must also catch mistakes it was not designed for.

5 checks → 6, tuned





References (IEEE style)

Classical models, policy and LLM-for-optimisation work

OPERATIONS RESEARCH · LLMs FOR OPTIMISATION

- [1] F. W. Harris, "How many parts to make at once," *Factory, The Magazine of Management*, vol. 10, no. 2, pp. 135–136, 1913. (Reprinted in *Operations Research*, vol. 38, no. 6, pp. 947–950, 1990.)
- [2] K. J. Arrow, T. Harris, and J. Marschak, "Optimal inventory policy," *Econometrica*, vol. 19, no. 3, pp. 250–272, 1951.
- [3] E. A. Silver, D. F. Pyke, and D. J. Thomas, *Inventory and Production Management in Supply Chains*, 4th ed. Boca Raton, FL, USA: CRC Press, 2017.
- [4] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.
- [5] Ministry of New and Renewable Energy, Govt. of India, "Approved Models and Manufacturers of Solar Photovoltaic Modules (Requirements for Compulsory Registration) Order, 2019," 2 Jan. 2019.
- [6] B. Li, K. Mellou, B. Zhang, J. Pathuri, and I. Menache, "Large language models for supply chain optimization," *arXiv:2307.03875*, 2023.
- [7] R. Ramamonjison et al., "NL4Opt competition: Formulating optimization problems based on their natural language descriptions," in *Proc. NeurIPS 2022 Competitions Track*, PMLR, vol. 220, pp. 189–203, 2022.
- [8] A. AhmadiTeshnizi, W. Gao, and M. Udell, "OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, 2024.
- [9] K. Cobbe et al., "Training verifiers to solve math word problems," *arXiv:2110.14168*, 2021.
- [10] F. Shi et al., "Large language models can be easily distracted by irrelevant context," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 202, pp. 31210–31227, 2023.



References (IEEE style)

Reliability of LLM outputs, tools and metrics, and models for Phase 2

LLM RELIABILITY · TOOLS · PHASE-2 MODELS

- [11] M. Xiong et al., "Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [12] J. Huang et al., "Large language models cannot self-correct reasoning yet," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [13] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Machine Learning (ICML), PMLR, vol. 70, pp. 1321–1330, 2017.
- [14] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020.
- [15] Anthropic, "Claude API documentation: Messages API," docs.anthropic.com (accessed Oct. 2026).
- [16] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [17] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.
- [18] H. M. Wagner and T. M. Whitin, "Dynamic version of the economic lot size model," Management Science, vol. 5, no. 1, pp. 89–96, 1958.
- [19] A. J. Clark and H. Scarf, "Optimal policies for a multi-echelon inventory problem," Management Science, vol. 6, no. 4, pp. 475–490, 1960.
- [20] Project source code: github.com/shreyasinha1502/Minor-Project (Python; benchmark, agent, verifier, scoring, dashboard).

Thank You

Project Summary

What we did, how we did it, and what is different about our approach



WHAT WE DID

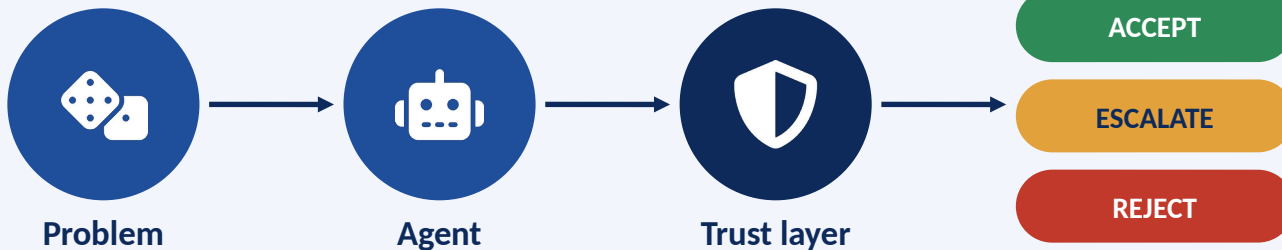
- Benchmark of **108 decisions** per run from 3 supply-chain families, with a verified answer key
- AI agent interface: offline simulator working, **real-LLM agent ready**
- **Five-check trust layer** plus scoring and a dashboard

HOW WE DID IT

- Closed-form OR results and an **LP solver** for the optimum
- Answer key re-derived by **grid search, Monte Carlo and random plans**
- Verifier sees **only** problem data + agent response

WHAT'S DIFFERENT

- Checks decisions **without an answer key**, at run time
- Rules come from **OR theory**, so every flag is explainable
- **100% of wrong answers flagged, 0% errors among accepted** (simulator, 4 seeds)



decided without the answer key

Thank You

Questions & Discussion

github.com/shreyasinha1502/Minor-Project